Numerical Analysis of Tridiagonal Systems

David Kaufman

March 17, 2026

Abstract

Tridiagonal systems are $n \times n$ square matrices with only nonzero entries on the main diagonal and the two adjacent diagonals. Because of their structured sparsity, they arise frequently in many different applied mathematical computations. This project investigates the structure, spectral properties, and numerical solution of linear systems involving tridiagonal matrices. After verifying a tridiagonal matrix admits LU factorization, a generalized formulation is derived for its entries of factors L and U for arbitrary matrix size n. A special case of a symmetric tridiagonal matrix is then analyzed in greater depth, for its eigenvalues and spectral radius. These results are applied to the solution of the linear system $Ax = b$ for a prescribed right-hand side vector b over a range of numerical methods. Gaussian elimination, LU factorization, and iterative algorithms including Jacobi, Gauss–Seidel, Jacobi Over-Relaxation (JOR), and Successive Over-Relaxation (SOR)—are implemented and compared using residual and error analysis to assess their accuracy, convergence behavior and computational performance.

Contents

1 Introduction **3**

2 Problem Description **3**

3 Problem 1 Results **5**

4 Problem 2 Results **7**

5 Problem 3 Results **7**

6 Problem 4 Results **9**

6.1 Gauss-Elimination 9

6.1.1 Without Pivoting 9

6.1.2 With Partial Pivoting 11

6.2 LU Factorization 12

6.3 Iterative Methods 14

6.3.1 Jacobi Method 15

6.3.2 Gauss-Seidel Method 17

6.3.3 Jacobi Over-Relaxation (JOR) 20

6.3.4 SOR Method 23

7 Conclusion **27**

8 Appendices **29**

8.1 MATLAB Code 29

1 Introduction

A Toeplitz matrix is a matrix where each descending diagonal from left to right is constant. An example of a Toeplitz matrix would be the following:

$$\begin{bmatrix} t_0 & t_1 & t_2 & t_3 \\ t_{-1} & t_0 & t_1 & t_2 \\ t_{-2} & t_{-1} & t_0 & t_1 \\ t_{-3} & t_{-2} & t_{-1} & t_0 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 2 & 6 \\ 4 & 1 & 3 & 2 \\ 5 & 4 & 1 & 3 \\ 7 & 5 & 4 & 1 \end{bmatrix} \quad ([1])$$

A tridiagonal matrix is a special form of a Toeplitz matrix where all non-zero elements are restricted to the main diagonal (b_i), subdiagonal (c_1), and super-diagonal (a_1) as shown below.

$$\mathrm{T} = \begin{bmatrix} b_1 & a_1 & 0 & 0 \\ c_1 & b_2 & a_2 & 0 \\ 0 & c_2 & b_3 & a_3 \\ 0 & 0 & c_3 & b_4 \end{bmatrix}, \qquad a_i,\ b_i,\ c_i \in \mathbb{R} \text{ for i = 1:4}$$

Their structure allows for efficient factorization and solution techniques. Understanding the factorization, spectral properties, and numerical behavior of tridiagonal systems is essential in the study of numerical methods. The notation $i : j$, is used to reference a sequence of numbers from i to j. An example would be: $1 : 4$, representing numbers 1, 2, 3, and 4. Another example would be $n : -1 : 1$ where it would represent numbers $n, \cdots, 2, 1$ by starting at the last number n and decrease by 1 up to the first entry (1).

2 Problem Description

This project will focus on analyzing tridiagonal matrices through several computational and theoretical approaches. Specifically, the tasks are as follows:

1. **LU Factorization Verification:**
   Verify the tridiagonal matrix T has the LU factorization given by L and U where

$$d_1 = b_1, d_{i+1} = b_{i+1} - \tfrac{a_i c_i}{d_i} \text{ for } i = 1 : 3$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & c_2/d_2 & 1 & 0 \\ 0 & 0 & c_3/d_3 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & d_2 & a_2 & 0 \\ 0 & 0 & d_3 & a_3 \\ 0 & 0 & 0 & d_4 \end{bmatrix},$$

2. **Generalized LU Factorization:**
   Describe the LU factorization for a tridiagonal matrix T of arbitrary size n and provide a generalized formula for the entries of L and U.

3. **Eigenvalues and Spectral Radius:**
   Considering the special tridiagonal matrix H defined by

$$H = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}, \quad a_i = c_i = -1 \ for \ i = 1 : n-1, \ b_i = 2 \ for \ i = 1 : n$$

   Show that the eigenvalues of H are

$$\lambda_k = 2 - 2\cos\left(\frac{k\pi}{n+1}\right), \quad k = 1 : n,$$

   and compute the spectral radius of H.

4. **Solving the Linear System:**
   For the system $H\mathbf{x} = \mathbf{b}$, where $\mathbf{b} = [1, 0, 0, \ldots, 0, 1]^T$, I will use the following methods for different values of n:

   (a) Gaussian elimination without and with partial pivoting,
   (b) LU factorization from Problem 2,
   (c) Jacobi iterative method,
   (d) Gauss-Seidel iterative method,
   (e) Jacobi Over-Relaxation (JOR) iterative method,
   (f) Successive Over-Relaxation (SOR) method with $w = 0.5, 1.0$, and 1.5, and estimate an optimal value for w.

   For the iterative methods, I will also compare the iterative methods' performance by analyzing their respective error measures:

$$\|\mathbf{x} - \tilde{\mathbf{x}}\|, \quad \|\mathbf{b} - A\tilde{\mathbf{x}}\|$$

   where the solution error norm $= ||x - \tilde{x}||$, $\tilde{\mathbf{x}}$ is the computed solution, and the residual norm $= ||b - Ax||$ and discuss the computation speed, accuracy, and stability for each method.

3 Problem 1 Results

LU factorization is a fundamental technique in linear algebra that allows efficient solution of linear systems by decomposing a matrix into a unit lower triangular matrix L and an upper triangular matrix U. Initially, the matrices are set as:

$$\text{L} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \text{U} = \begin{bmatrix} b_1 & a_1 & 0 & 0 \\ c_1 & b_2 & a_2 & 0 \\ 0 & c_2 & b_3 & a_3 \\ 0 & 0 & c_3 & b_4 \end{bmatrix}$$

To prove matrix T is LU factorable, row operations such as the following will be conducted.

1. Interchange two rows ($R_i \iff R_j$)
2. Multiply a row by a nonzero constant ($R_i = \alpha R_i, \alpha \neq 0$)
3. Adding a multiple of a row to another row ($R_i = R_i + \alpha R_j$)

The third row operation will be used frequently to transform matrix T it into upper triangular form (a matrix with only non-zero entries at or above the main diagonal) by the Gaussian-Elimination (GE) process. For the GE process, it would start in the leftmost column in the first diagonal entry (T_{11}) and move until the second to last diagonal entry ($T_{n-1,n-1}$).

Gaussian elimination steps:

1. Let $d_1 = b_1$ and eliminate c_1 below d_1:

$$R_2 \to R_2 - \frac{c_1}{d_1} R_1, \quad L_{21} = \frac{c_1}{d_1}$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & b_2 - \frac{c_1}{d_1} a_1 & a_2 & 0 \\ 0 & c_2 & b_3 & a_3 \\ 0 & 0 & c_3 & b_4 \end{bmatrix}$$

$$\text{Let } d_2 = b_2 - \tfrac{c_1}{d_1} a_1$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & d_2 & a_2 & 0 \\ 0 & c_2 & b_3 & a_3 \\ 0 & 0 & c_3 & b_4 \end{bmatrix}$$

2. Eliminate c_2 below d_2:

$$R_3 \to R_3 - \frac{c_2}{d_2} R_2, \quad L_{32} = \frac{c_2}{d_2}$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & c_2/d_2 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & d_2 & a_2 & 0 \\ 0 & 0 & b_3 - \frac{c_2}{d_2} a_2 & a_3 \\ 0 & 0 & c_3 & b_4 \end{bmatrix}$$

$$\text{Let } d_3 = b_3 - \tfrac{c_2}{d_2} a_2$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & c_2/d_2 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & d_2 & a_2 & 0 \\ 0 & 0 & d_3 & a_3 \\ 0 & 0 & c_3 & b_4 \end{bmatrix}$$

3. Eliminate c_3 below d_3:

$$R_4 \to R_4 - \frac{c_3}{d_3} R_3, \quad L_{43} = \frac{c_3}{d_3}$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & c_2/d_2 & 1 & 0 \\ 0 & 0 & c_3/d_3 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & d_2 & a_2 & 0 \\ 0 & 0 & d_3 & a_3 \\ 0 & 0 & 0 & b_4 - \frac{c_3}{d_3} a_3 \end{bmatrix}$$

$$\text{Let: } d_4 = b_4 - \tfrac{c_3}{d_3} a_3$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & c_2/d_2 & 1 & 0 \\ 0 & 0 & c_3/d_3 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & d_2 & a_2 & 0 \\ 0 & 0 & d_3 & a_3 \\ 0 & 0 & 0 & d_4 \end{bmatrix}$$

Verification: To confirm the factorization, multiply L and U:

$$LU = \begin{bmatrix} 1 & 0 & 0 & 0 \\ c_1/d_1 & 1 & 0 & 0 \\ 0 & c_2/d_2 & 1 & 0 \\ 0 & 0 & c_3/d_3 & 1 \end{bmatrix} \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ 0 & d_2 & a_2 & 0 \\ 0 & 0 & d_3 & a_3 \\ 0 & 0 & 0 & d_4 \end{bmatrix} = \begin{bmatrix} d_1 & a_1 & 0 & 0 \\ c_1 & \frac{c_1 a_1}{d_1} + d_2 & 0 & 0 \\ 0 & c_2 & \frac{c_2 a_2}{d_2} + d_3 & a_3 \\ 0 & 0 & c_3 & \frac{c_3 a_3}{d_3} + d_4 \end{bmatrix}$$

Since I let $d_1 = b_1, \quad d_{i+1} = b_{i+1} + \frac{a_i c_i}{d_i}$ for $i = 2:4$, by reversing the order:

$$b_1 = d_1 \quad b_{i+1} = d_{i+1} - \frac{a_i c_i}{d_i} \quad \text{(for i = 2:4)}$$

The matrix will transform into the initial T matrix and T is LU factorable for any tridiagonl matrix of size n with with $d_1 = b_1$ and $d_{i+1} = b_{i+1} - \frac{a_i c_i}{d_i}, i = 1:n$

$$T = \begin{bmatrix} b_1 & a_1 & 0 & 0 \\ c_1 & b_2 & 0 & 0 \\ 0 & c_2 & b_3 & a_3 \\ 0 & 0 & c_3 & b_4 \end{bmatrix}$$

4 Problem 2 Results

Building on the LU factorization from in Problem 1, I can generalize the factorization for a tridiagonal matrix T of arbitrary size n:

$$T = \begin{bmatrix} b_1 & a_1 & 0 & 0 & \cdots & 0 \\ c_1 & b_2 & a_2 & 0 & \cdots & 0 \\ 0 & c_2 & b_3 & a_3 & \cdots & 0 \\ 0 & 0 & c_3 & b_4 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & a_{n-1} \\ 0 & 0 & 0 & 0 & c_{n-1} & b_n \end{bmatrix}_{n\times n},$$

The corresponding general form of the LU factorization for T is:

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ \frac{c_1}{d_1} & 1 & 0 & 0 & \cdots & 0 \\ 0 & \frac{c_2}{d_2} & 1 & 0 & \cdots & 0 \\ 0 & 0 & \frac{c_3}{d_3} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & 0 \\ 0 & 0 & 0 & 0 & \frac{c_{n-1}}{d_{n-1}} & 1 \end{bmatrix}, U = \begin{bmatrix} b_1 & a_1 & 0 & 0 & \cdots & 0 \\ 0 & b_2 & a_2 & 0 & \cdots & 0 \\ 0 & 0 & b_3 & a_3 & \cdots & 0 \\ 0 & 0 & 0 & b_4 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & a_{n-1} \\ 0 & 0 & 0 & 0 & 0 & b_n \end{bmatrix}$$

$$\text{where } b_1 = d_1 \text{ and } b_{i+1} = d_{i+1} - \tfrac{a_i c_i}{d_i} \text{ for } i = 2:n$$

For a specific case used in later problems, I will set $b_i = 2$ and $a_i = c_i = -1$ for $i = 1:n$. This produces the tridiagonal matrix H of size $n \times n$:

$$H = \begin{bmatrix} 2 & -1 & 0 & 0 & \cdots & 0 \\ -1 & 2 & -1 & 0 & \cdots & 0 \\ 0 & -1 & 2 & -1 & \cdots & 0 \\ 0 & 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & 0 & -1 & 2 \end{bmatrix}_{n\times n}$$

5 Problem 3 Results

From the *Matrix Analysis and Applied Linear Algebra* by Carl Meyer, the eigenvalues of a tri-diagonal matrix T are given by:

$$\lambda_i = b_i + 2a_j\sqrt{\tfrac{c_j}{a_j}}cos(\tfrac{k\pi}{n+1}) \quad \text{for } k = 1, ..., n \text{ ([1])}$$

Considering the matrix H where entries are: $b_i = 2, a_i = c_i = -1$ for $i = 1:n$, the eigenvalue formula becomes:

$$\lambda_i = 2 - 2\cos\left(\frac{k\pi}{n+1}\right), \quad k = 1:n.$$

Using the formula, I can calculate the spectral radius ($\rho(H)$ = the largest absolute eigenvalue), while considering $n = 4$. As i increases, the eigenvalues would peak at the last value when $n = 4$. If I consider the next set of i values, say $k = 5 : 8$, the calculations peak at λ_5 then decreases with the same respective values of λ_4, λ_3 and λ_2. This matches fits to the bounded nature of the cosine function on $[0, 2\pi]$ As cosine increases from $[0, \pi]$ but then decreases from $[\pi, 2\pi]$.

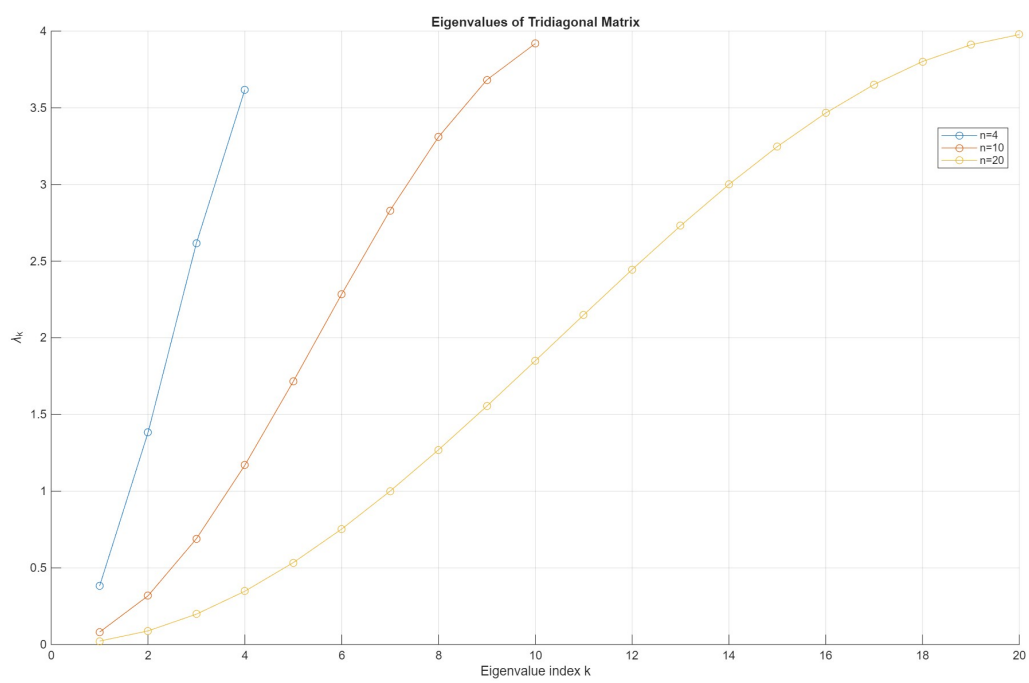
Table 1: Spectral Radius of H

i	1	2	3	4	5	6	7	8
λ_i	0.38	1.38	2.62	3.62	4	3.62	2.62	1.38

To further understand, a provided graph depicts the eigenvalues for $1 : n$ for matrices size $n = 4, 10$ and 20. For the spectral radius, it would be the maximum of each of these values which are when $i = n$. Their respective spectral radii are given as follows:

Table 2: Spectral Radius of H of size n

n	4	10	20
λ_i	3.62	3.92	3.98



In general, for a tridiagonal matrix H of size n, the spectral radius is:

$$\rho(H) = \lambda_i = 2 - 2cos(\frac{n\pi}{n+1})$$

Using the trigonometric identity $\cos(a-b) = cos(a)cos(b) + sin(a)sin(b)$, the spectral radius can further derived as:

$$\begin{aligned}
\lambda_{max} &= 2 - 2cos(\frac{n\pi}{n+1}) \\
&= 2 - 2cos(\pi - \frac{\pi}{n+1}) \qquad \text{(cosine identity)} \\
&= 2 + cos(\frac{\pi}{n+1}) \qquad \text{(simplify)}
\end{aligned}$$

So the spectral radius for the tridiagonal matrix H of size $n \times n$ is: $\rho(H) = 2 + 2cos(\frac{n\pi}{n+1})$ for arbitrary size n.

6 Problem 4 Results

For each method, I will consider the system $Hx = b$, where H is the described matrix from Problem 2 and b is the right-hand side vector b where:

$$b = [1, 0, 0, ..., 0, 1]^T_{n\times 1} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}_{n\times 1}$$

6.1 Gauss-Elimination

As previously mentioned in Problem 1, Gaussian elimination (GE) transforms a matrix into upper-triangular form, and is solved via back substitution. I will apply the GE process to H with and without partial pivoting.

6.1.1 Without Pivoting

For matrix size $n = 4$. H and b are:

$$H = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Applying GE (row operations) transforms H into upper-triangular form U while also transforming b.

$$\text{Hx} = \text{Ux} = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}_{4\times 4} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \text{b}$$

$$\left[\begin{array}{cccc|c} 2 & -1 & 0 & 0 & 1 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & 1 \end{array}\right]$$

Row operation 1: $R_2 = R_2 - \frac{1}{2}R_1$;

$$\left[\begin{array}{cccc|c} 2 & -1 & 0 & 0 & 1 \\ 0 & \frac{3}{2} & -1 & 0 & 1/2 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & 1 \end{array}\right]$$

Row operation 2: $R_3 = R_3 - \frac{2}{3}R_2$;

$$\left[\begin{array}{cccc|c} 2 & -1 & 0 & 0 & 1 \\ 0 & \frac{3}{2} & -1 & 0 & 1/2 \\ 0 & 0 & \frac{4}{3} & -1 & 1/3 \\ 0 & 0 & -1 & 2 & 1 \end{array}\right]$$

Row operation 3: $R_4 = R_4 - \frac{3}{4}R_3$;

$$\left[\begin{array}{cccc|c} 2 & -1 & 0 & 0 & 1 \\ 0 & \frac{3}{2} & -1 & 0 & 1/2 \\ 0 & 0 & \frac{4}{3} & -1 & 1/3 \\ 0 & 0 & 0 & \frac{5}{4} & 5/4 \end{array}\right]$$

When solving for solution vector x, it would transform the above form in to a system of linear equations:

$$\begin{aligned} 2x_1 - x_2 + 0x_3 + 0x_4 &= 1 \\ \tfrac{3}{2}x_2 - x_3 + 0x_4 &= \tfrac{1}{2} \\ \tfrac{4}{3}x_3 - x_4 &= \tfrac{1}{3} \\ \tfrac{5}{4}x_4 &= \tfrac{5}{4} \end{aligned}$$

Then by the back substitution (solving for x_i, $i = n : -1 : 1$) the solution vector entries become:

$$\mathrm{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

For general size n, the same procedure produces a similar system of linear equations and solution vector x of n ones.

$$\mathrm{Hx} = \mathrm{Ux} = \begin{bmatrix} 2 & -1 & 0 & 0 & \cdots & 0 \\ 0 & \frac{3}{2} & -1 & 0 & \cdots & 0 \\ 0 & 0 & \frac{4}{3} & -1 & \cdots & 0 \\ 0 & 0 & 0 & \frac{5}{4} & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & 0 & 0 & \frac{n+1}{n} \end{bmatrix}_{n\times n} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} 1 \\ 1/2 \\ 1/3 \\ 1/4 \\ \vdots \\ \frac{n+1}{n} \end{bmatrix} = \mathrm{b};$$

$$
\begin{aligned}
2x_1 - x_2 + 0x_3 + 0x_4 + \cdots + 0x_n &= 1, \\
\tfrac{3}{2}x_2 - x_3 + 0x_4 + \cdots + 0x_n &= \tfrac{1}{2} \\
\tfrac{4}{3}x_3 - x_4 + \cdots + 0x_n &= \tfrac{1}{3} \\
\tfrac{5}{4}x_4 + \cdots + 0x_n &= \tfrac{1}{3} \\
\ddots \quad \vdots \quad &= \vdots \\
\tfrac{n+1}{n}x_n &= \tfrac{n+1}{n}
\end{aligned}
$$

Solution vector of n size vector of 1's:

$$
x = \mathbf{1}_{n\times 1} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}_{n\times 1}
$$

6.1.2 With Partial Pivoting

Partial pivoting swaps rows so that the pivot element (H_{ii} for $i = 1 : n-1$) is the largest absolute value in its column. For a general matrix A, this improves numerical stability. An example would the matrix A on the next page where for column 1, the largest absolute entry is 23 which is in row 2 (R_2). Row swapping would occur between rows 1 and 2, $R_1 \Longleftrightarrow R_2$ where the resulting matrix is A_{new} which highlights the change in matrix A.

$$
\text{A} = \begin{bmatrix} 17 & 24 & 1 & 8 & 15 \\ 23 & 5 & 7 & 14 & 16 \\ 4 & 6 & 13 & 20 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 18 \end{bmatrix}; A_{new} = \begin{bmatrix} 23 & 5 & 7 & 14 & 16 \\ 17 & 24 & 1 & 8 & 15 \\ 4 & 6 & 13 & 20 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 18 \end{bmatrix} \; (R_1 \Longleftrightarrow R_2)
$$

However, for H, no row swaps are required because it is diagonally dominant:

$$
|H_{ii}| \geq \sum_{j\neq i} |H_{ij}|, \quad i = 1 : n
$$

This implies the absolute row sum of the non-diagaonal entries equals their respective diagonal entry. If I consider the matrix H of size $n = 4$,

$$
|H_{11}| = |2| \geq \sum_{\substack{j=1 \\ j>i}}^{4} |H_{1j}| = |-1| + |0| + |0| = 1 \quad (\text{i} = 1)
$$

$$
|H_{22}| = |2| \geq \sum_{\substack{j=1 \\ j\neq i}}^{4} |H_{2j}| = |-1| + |-1| + |0| = 2 \quad (\text{i} = 2)
$$

$$|H_{33}| = |2| \geq \sum_{\substack{j=1 \\ j\neq i}}^{4} |H_{3j}| = |0| + |-1| + |-1| = 2 \quad \text{(i = 3)}$$

$$|H_{44}| = |2| \geq \sum_{\substack{j=1 \\ j\neq i}}^{4} |H_{4j}| = |0| + |0| + |-1| = 1 \quad \text{(i = 4)}$$

Hence, partial pivoting produces the same upper-triangular U as GE without pivoting for arbitrary size n. The $Hx = b$ would be the same system of equations on page 9 and the solution vector remains the same without pivoting. So with partial pivoting, the GE process would be a ones vector of size n.

$$x = \mathbf{1}_{n\times 1} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}_{n\times 1}$$

6.2 LU Factorization

To solve the linear system $Hx = b$, performing LU factorization on matrices of arbitrary size n decomposes H into a lower-triangular matrix L and an upper-triangular matrix U such that: $H = LU$. The solution proceeds in two steps:

$$\begin{aligned} Hx &= b \\ (LU)x &= b \quad \text{(H = LU)} \\ Ly &= b \quad \text{(y = Ux)} \end{aligned}$$

A user would solve by the following forward substitution method for the intermediate vector y from $Ly = b$:

$$y_1 = \tfrac{b_1}{L_{11}}, y_i = \tfrac{b_i - \sum_{k=i+1}^{n} L_{ik} y_k}{L_{ii}} \text{ for } i = 2 : n,$$

then for back substitution method for solution vector x from $Ux = b$.

$$x_n = \frac{y_n}{U_{nn}}, x_i = \frac{y_i - \sum_{k=i+1}^{n} U_{ik} x_k}{U_{ii}}, \quad i = n-1 : 1.$$

For $n = 4$, forward substitution for $Ly = b$ would correspond to a system of linear equations.

$$\text{Ly} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 & 0 \\ 0 & -\frac{2}{3} & 1 & 0 \\ 0 & 0 & -\frac{3}{4} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$
\begin{aligned}
y_1 + 0y_2 + 0y_3 + 0y_4 &= 1 \\
-\tfrac{1}{2}y_1 + y_2 + 0y_3 + 0y_4 &= 0 \\
-\tfrac{2}{3}y_2 + y_3 + 0y_4 &= 0 \\
-\tfrac{3}{4}y_3 + y_4 &= 1
\end{aligned}
$$

The intermediate vector y would become:

$$
\mathrm{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1/2 \\ 2/3 \\ 5/4 \end{bmatrix}
$$

By back substitution, it would be the same pattern as in Gaussian-elimination resulting into a vector of all ones.

$$
\mathrm{Ux} = \begin{bmatrix} 2 & -1 & 0 & 0 \\ 0 & \frac{3}{2} & -1 & 0 \\ 0 & 0 & \frac{4}{3} & -1 \\ 0 & 0 & 0 & \frac{5}{4} \end{bmatrix}_{4\times 4} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1/2 \\ 1/3 \\ 5/4 \end{bmatrix} = \mathrm{b}
$$

$$
\begin{aligned}
2x_1 - x_2 + 0x_3 + 0x_4 &= 1 \\
\tfrac{3}{2}x_2 - x_3 + 0x_4 &= \tfrac{1}{2} \\
\tfrac{4}{3}x_3 - x_4 &= \tfrac{1}{3} \\
\tfrac{5}{4}x_4 &= \tfrac{5}{4}
\end{aligned}
$$

$$
\mathrm{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}
$$

In general, the forward and backward substitution for LU factorization with the intermediate vector y and solution vector x would be on the following for arbitrary size n.

Forward Substitution:

$$
\mathrm{Ly} = \begin{bmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ -\frac{1}{2} & 1 & 0 & 0 & \cdots & 0 \\ 0 & -\frac{2}{3} & 1 & 0 & \cdots & 0 \\ 0 & 0 & -\frac{3}{4} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -\frac{n+1}{n} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} = \mathrm{b}
$$

$$\text{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 \\ 1/2 \\ 2/3 \\ 5/4 \\ \vdots \\ (n+1)/n \end{bmatrix}$$

Back Substitution:

$$\text{Ux} = \begin{bmatrix} 2 & -1 & 0 & 0 & \cdots & 0 \\ 0 & \frac{3}{2} & -1 & 0 & \cdots & 0 \\ 0 & 0 & \frac{4}{3} & -1 & \cdots & 0 \\ 0 & 0 & 0 & \frac{5}{4} & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & 0 & 0 & \frac{n+1}{n} \end{bmatrix}_{n\times n} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} 1 \\ 1/2 \\ 1/3 \\ 1/4 \\ \vdots \\ \frac{n+1}{n} \end{bmatrix} = \text{y}$$

$$\text{x} = 1_{n\times 1} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \\ 1 \end{bmatrix}_{n\times 1}$$

6.3 Iterative Methods

In this section, I will apply the iterative methods — **Jacobi**, **Gauss-Seidel**, **Jacobi Over-Relaxation (JOR)**, and **Successive Over-Relaxation (SOR)** — to approximate the solution vector x for the linear system $Hx = b$. Each method constructs a sequence of approximations:

$$x^{(0)}, x^{(1)}, x^{(2)}, \ldots, x^{(k)}$$

that take a general form:

$$x^{(k+1)} = Bx^{(k)} + c,$$

where B is the iteration matrix and c is a constant vector. Starting from an initial guess $x^{(0)} = b$, I iterate until convergence. For example, consider a 3×3 system:

$$Ax = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = b,$$

or equivalently as three equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned}$$

In practice, convergence is assessed using

- the **error norm**: $\|x - \tilde{x}\|$, where x is the true solution and $\tilde{x}$ is approximate;
- the **residual norm**: $\|r^{(k)}\| = \|b - Hx^{(k)}\|$.

Both are reported for each method to compare overall accuracy and stability where the $||x||$ notation refers to the 2-norm formula:

$$\|x\|_2 = (\sum_{i=1}^{n} |x_i|^2)^{1/2}$$

Each $\tilde{x}$ and residual norm takes the squared sum of each value and square roots the total.

6.3.1 Jacobi Method

To solve by the Jacobi Method by the $Ax = b$ example would be for the $x_i^{(k+1)}$ coefficients for $i = 1 : 3$.

$$\begin{aligned} x_1^{(k+1)} &= \frac{b_i - a_{12}x_2^{(k)} + a_{13}x_3^{(k)}}{a_{11}} \\ x_2^{(k+1)} &= \frac{b_2 - a_{21}x_1^{(k)} + a_{23}x_3^{(k)}}{a_{22}} \\ x_3^{(k+1)} &= \frac{b_3 - a_{31}x_1^{(k)} + a_{32}x_2^{(k)}}{a_{33}} \end{aligned}$$

For an arbitrary size n, the formula generalizes to:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\Big) \quad i = 1 : n$$

By decompose A as

$$A = D - L - U,$$

where D is the diagonal, L is strictly lower triangular, and U is strictly upper triangular. The Jacobi iteration takes the matrix form:

$$Dx^{(k+1)} = b + (L + U)x^{(k)} \quad \Longrightarrow \quad x^{(k+1)} = D^{-1}b + D^{-1}(L + U)x^{(k)},$$

with iteration matrix and constant vector

$$B_J = D^{-1}(L + U), \quad c_J = D^{-1}b$$

Jacobi was tested for $n = 4$, $n = 20$, and the largest n that converged before the iteration limit was reached. Results are shown in Table 3 with the final error and residual norms for the three matrix sizes.

Jacobi converges for small and moderately sized systems, but requires many iterations. For $n = 4$, the method converges in 112 iterations whereas $n = 20$, the number of iterations grows significantly. However the approximation breaks when $n = 27$ due to failing to converge

Table 3: Jacobi Method Performance

n	**Iteration Number**	**Final** $\|x^{(k)}\|$	**Final** $\|r^{(k)}\|$
4	112	$9.56 * 10^{-11}$	$3.65 * 10^{-11}$
20	2177	$9.89 * 10^{-11}$	$2.21 * 10^{-12}$
27	3898	$9.9863 * 10^{-11}$	$1.78 * 10^{-12}$

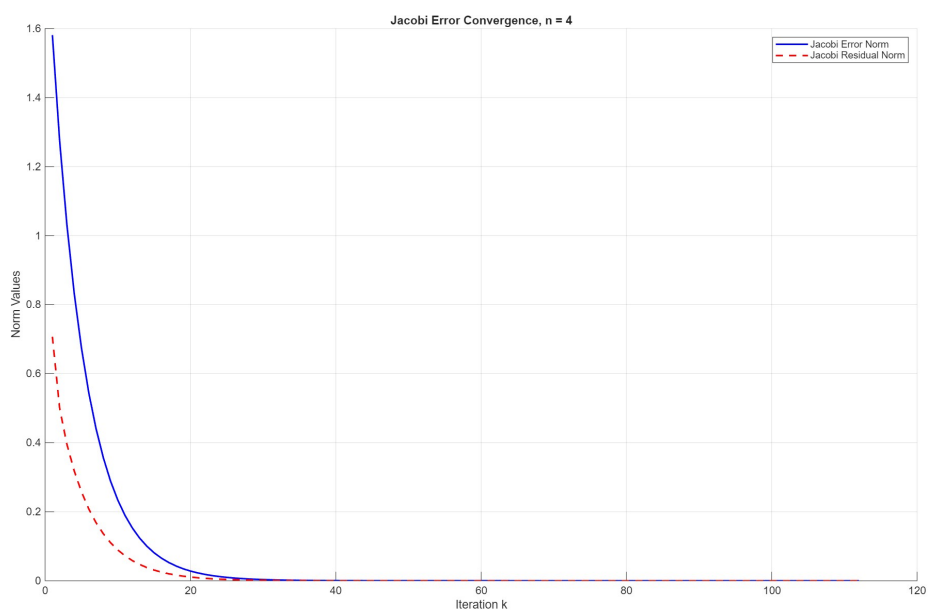
within the iteration limit. For the true solution x to the $Hx = b$, the approximated solution vector $\tilde{x}$ by the Jacobi Method, in each convergent case has an approximation within the 10^{-10} of the true solution $x = \mathbf{1}_{n\times 1}$. A typical form of the final approximation (here shown for $n = 20$) is a vector of approximations close or are ones whereas the true solution vector x is a ones vector of size n.

$$\tilde{x}^{(k)} \approx \left[0.999999999998425, 0.999999999996889, \cdots, 0.999999999998425\right]^T_{20\times 1}$$

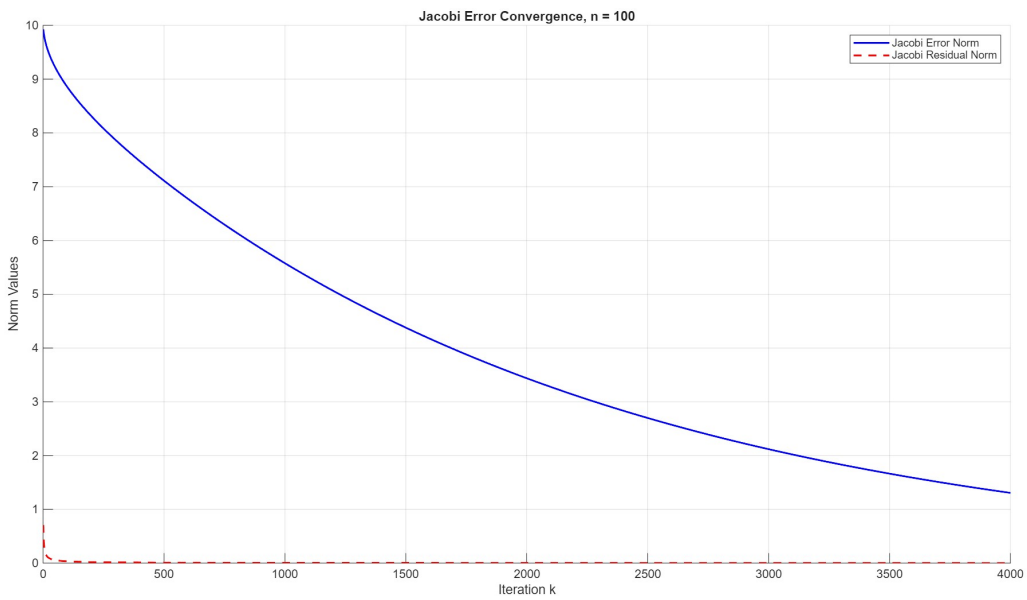
$$x = \left[1, 1, \cdots, 1\right]^T_{n\times 1} \quad \text{(Jacobi true solution } x\text{)}$$

For any non-convergent it would have the approximations that would fail to be with in the 10^{-10} convergence. An example would be the approximate solution vector for H of size $n = 40$ shown below. For the approximation we see that the output is close to 1 but the digits are not within the the precison as the true solution shown above. These numbers are shown in bold.

$$\tilde{x}^{(k)} \approx \left[0.99999\mathbf{234543883},\ 0.99999\mathbf{8473579767},\ \cdots,\ 0.99999\mathbf{234543883}\right]^T_{40\times 1}$$



As for the final error solution $\|x^{(k)}\|$ and residual norm $\|r^{(k)}\|$ from Table 3, it gives an indication they appear to be close to each other. When plotted using a semiology plot to visualize the decay in error, the error-norm and residual-norm plots decrease slowly in natural log (ln) pattern when plotted side-by-side on a semilogy plot. However there is an initial gap between the lines indicating the residual is not always the best estimation of error. As the method starts from iteration 0 to if it does converge, the errors keep close to zero to the point it visually looks like they are the same. As n increases, the residual norm get into the more of an L-shaped due to the matrix converging using previous $\tilde{x}$ approximations. This is due to the increasing size of the matrix and the method being inefficient for the solution error as n increases and failing to converge beyond $n = 27$ within the allowed iteration limit. For a non-convergent case would be when $n = 100$ as the solution error norm is still taking time to converge to ϵ



6.3.2 Gauss-Seidel Method

If I solve the system of equations again but for $x_i^{(k+1)}$ the system of linear equations would give the following for the prior example:

$$x_i^{(k+1)} = \frac{b_i - a_{12}x_2^{(k)} + a_{13}x_3^{(k)}}{a_{11}}$$
$$x_2^{(k+1)} = \frac{b_2 - a_{21}x_1^{(k+1)} + a_{23}x_3^{(k)}}{a_{22}}$$
$$x_3^{(k+1)} = \frac{b_3 - a_{31}x_1^{(k+1)} + a_{32}x_2^{(k)}}{a_{33}}$$

For an arbitrary size n, the formula generalizes to:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\Big), \quad i = 1:n.$$

In matrix form:

$$(D-L)x^{(k+1)} = b + Ux^{(k)} \quad \Longrightarrow \quad x^{(k+1)} = (D-L)^{-1}b + (D-L)^{-1}Ux^{(k)},$$

so the iteration matrix B and constant vector c for Gauss-Seidel are:

$$B_{GS} = (D-L)^{-1}U, \quad c_{GS} = (D-L)^{-1}b$$

For the approximated solution vector $\tilde{x}$ by the Gauss-Seidel Method, in each convergent case it has the same output as in Jacobi. The has an approximation if converged is within the 10^{-10} of the true solution $x = \mathbf{1}_{n\times 1}$. For $n = 20$, the approximated vector is roughly close to a vector of ones. For any non-convergent it would have the approximations that would fail to be with in the 10^{-10} convergence which would be the same as the nonconvergent case in Jacobi (page 13).

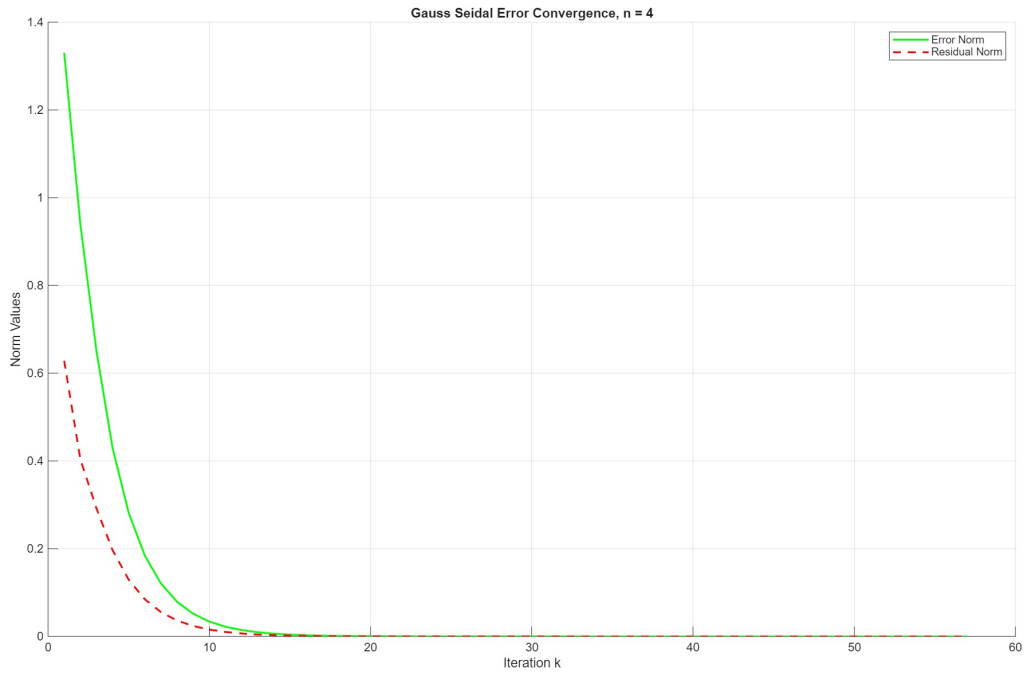
$$\tilde{x}^{(k)} \approx \big[0.999999999999999, 0.999999999999999, \cdots, 0.999999999999999\big]^T_{20\times 1}$$

$$x = \big[1, 1, \cdots, 1\big]^T_{n\times 1} \quad \text{(Gauss-Sediel true solution } x\text{)}$$

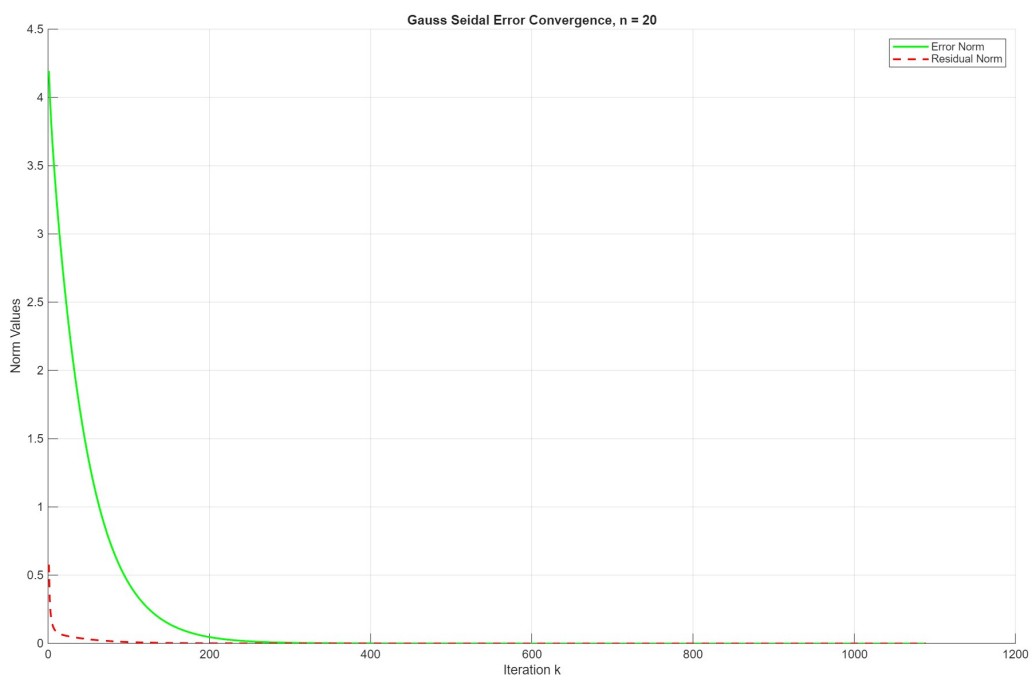
Using the same matrix sizes for Jacobi method, the Gauss-Sediel results are shown on the next page for matrices of size $n = 4, 20$ and n. Comparing the results of Gauss-Seidel (Table 4) to Jacobi (Table 3), the Gauss Seidel method converges faster than Jacobi. This is because it uses updated values of $x^{(k)}$ immediately in the computation of $x^{(k+1)}$. For $n = 4$, the method requires only 57 iterations, roughly half of the Jacobi method. For $n = 20$, the number of iterations increases to 1089, but the error norms remains almost the same. Even for $n = 100$, where convergence is not achieved, the solution error is significantly smaller than that of Jacobi, indicating that Gauss-Seidel is more efficient and accurate for larger systems.

Table 4: Gauss-Seidel Method Performance

n	**Iteration Number**	**Final** $\|x^{(k)}\|$	**Final** $\|r^{(k)}\|$
4	57	$7.54 * 10^{-11}$	$3.46 * 10^{-11}$
20	1089	$9.91 * 10^{-11}$	$2.25 * 10^{-12}$
38	3811	$1.00 * 10^{-12}$	$6.52 * 10^{-13}$



For the graphical depictions of the error and residual norms themselves, their behavior has the same behavior with the Jacobi method. The only difference between them is the number of iterations it converges and the norm difference. The starting norm values for each of the solution vector x and residual r are slightly smaller than Jacobi which makes sense because Gauss-Seidel is a better approximation than Jacobi.



6.3.3 Jacobi Over-Relaxation (JOR)

For the JOR method, suppose I consider the Jacobi method but with a relaxation parameter w and residual $r_i^{(k)}$ as follows:

$$x_i^{(k+1)} = x_i^{(k)} + \frac{w}{a_{ii}} r_i^{(k)}, \quad r_i^{(k)} = b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^{n} a_{ij} x_j^{(k)}$$

The relaxation parameter w acts as a scalar for the JOR method where if:
- $w < 1$, the method undergoes Under-relaxation leading to slower convergence
- $w > 1$, the method undergoes Over-relaxation leading to faster convergence

In matrix form, JOR becomes:

$$Dx^{(k+1)} = (w(L+U)+(1-w)D)x^{(k)}+wb \quad \Longrightarrow \quad x^{(k+1)} = D^{-1}[w(L+U)+(1-w)D]x^{(k)}+wD^{-1}b$$

where the iteration matrix and constant vector are:

$$B_{JOR} = D^{-1}[w(L+U) + (1-w)D]$$
$$c_{JOR} = wD^{-1}b$$

For the approximated solution vector $\tilde{x}$ by the JOR Method, in each convergent case it has the same true x as in regular Jacobi.

$$\text{x} = \left[1, 1, \cdots, 1\right]_{n\times 1}^T \quad \text{(JOR true solution } x\text{)}$$
$$\tilde{x}^{(k)} \approx \left[1.000000000000000, 0.999999999999999, 0.999999999999999, 1.000000000000000\right]_{4\times 1}^T$$

With the relaxation parameter w, JOR noticeably converges less than the Jacobi or Gauss-Sediel method. Given by the (Table 5) when $w = 0.5$, it converges is within the 10^{-10} of the true solution $x = \mathbf{1}_{n\times 1}$ but for a small matrix H size of n.

Table 5: JOR Method Performance

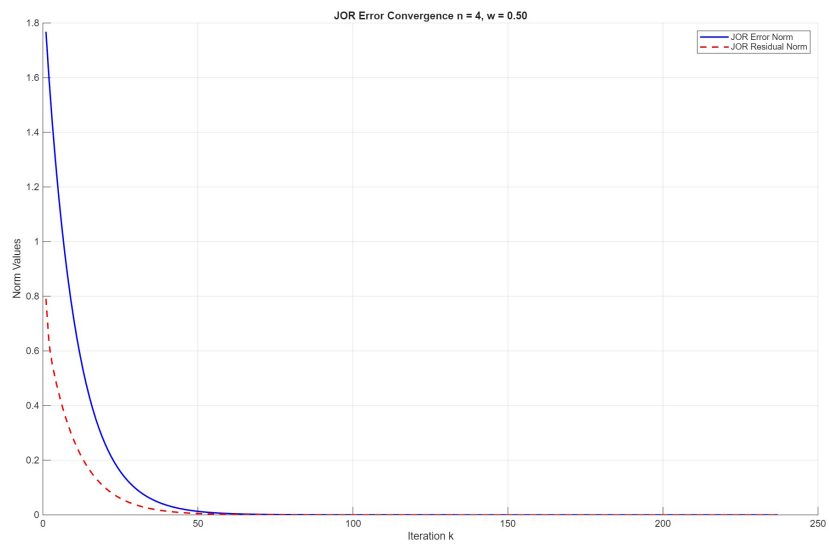
n	w	**Iteration Number**	**Final** $\|x^{(k)}\|$	**Final** $\|r^{(k)}\|$
4	0.5	237	$9.10 * 10^{-11}$	$3.48 * 10^{-11}$
19	0.5	3955	$9.951 * 10^{-11}$	$2.45 * 10^{-12}$
20	0.5	N/A	$7.69 * 10^{-10}$	$1.72 * 10^{-11}$
4	1.0	112	$9.56 * 10^{-11}$	$3.65 * 10^{-11}$
20	1.0	2177	$9.89 * 10^{-11}$	$2.21 * 10^{-12}$
27	1.0	3898	$9.99 * 10^{-11}$	$1.78 * 10^{-12}$
4	1.5	599	$9.50 * 10^{-11}$	$2.44 * 10^{-12}$
20	1.5	N/A	NaN	NaN

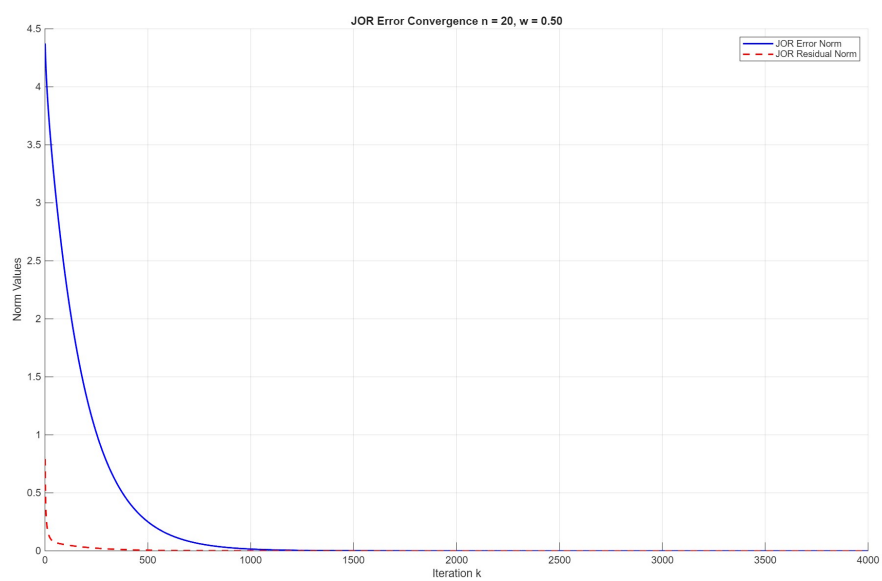
When $n = 20$, the approximated vector has values of NaN. This result usually happens when the values are not real numbers or integers. This approximation appears to have numbers that are not integers. When $w = 1.5$, it fully takes on NaN values which would highlight that JOR would not converge at all for $w > 1$.

$$\tilde{x}^{(k)} \approx \left[0.999999999999999, \cdots, Inf, \cdots, NaN, NaN\right]^T_{n\times 1} \quad (\text{n} \geq 4)$$

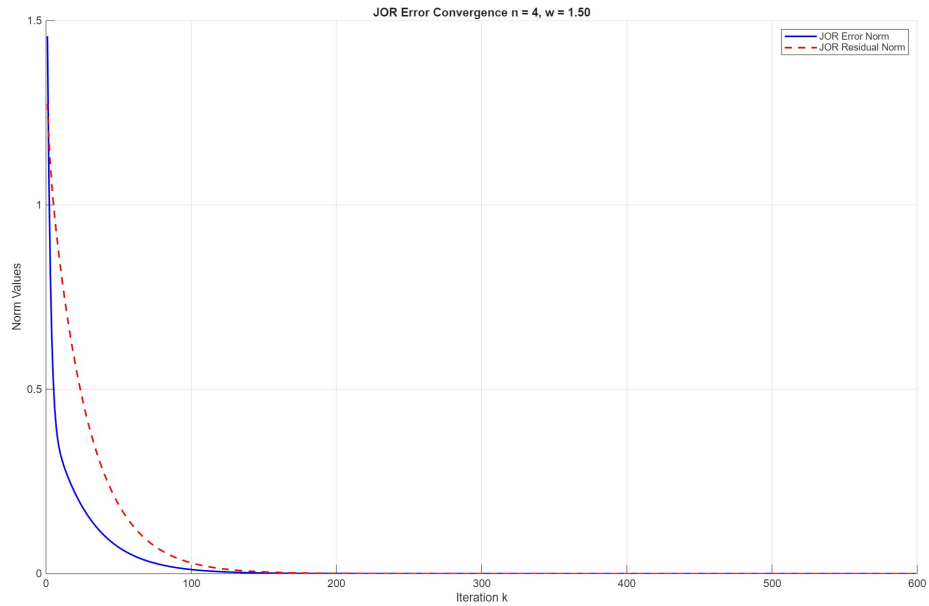
$$\tilde{x}^{(k)} \approx \left[NaN, NaN, \cdots, NaN, NaN\right]^T_{20\times 1} \quad (w = 1.5)$$

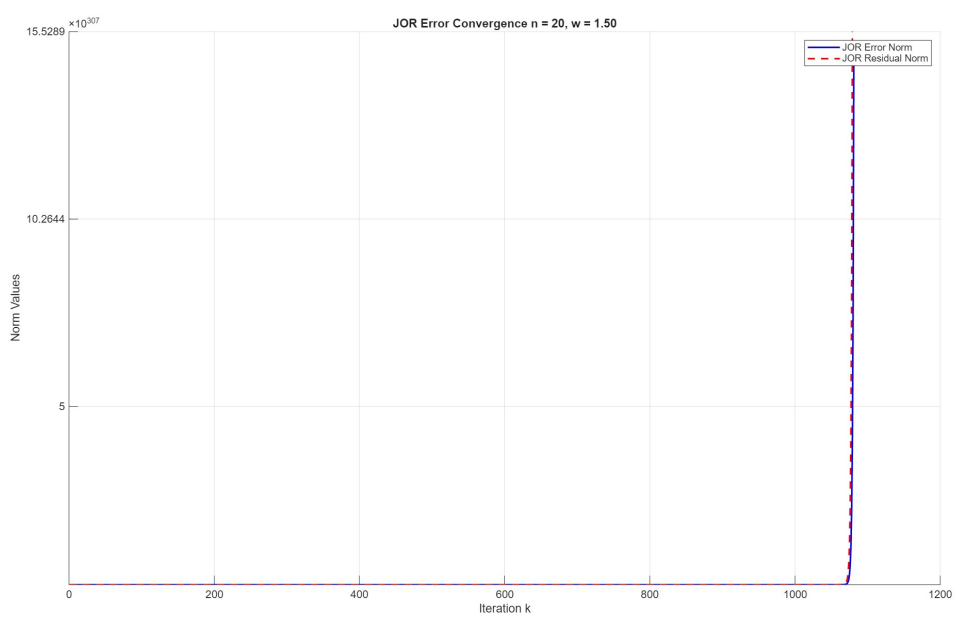
For error and residual norm plots, JOR method exhibits very different convergence rates with the relaxation parameter w. There is still that logarithmic depiction where the error norm is above the residual norm and then it converges but as n increases, the initial gap between them increases until it diverges obscurely. Since it is based off the Jacobi method, the failed convergence rate led to obscure outputs. When $w < 1$, the error and residual norm plots appear to have the same representation as Jacobi and Gauss-Sediel but for a larger size n say $n = 20$, it converges either faster or diverges quickly.





As for when the $w = 1.5$, there is a huge difference in terms of depicting the $||x||$ and $||r||$. Usually Jacobi and Gauss-Sediel each still have sometime or are already finished by converging but for JOR, the solution error norm is smaller than the residual norm. It starts off initially above the estimate but then both appear to be converging. For the diverging case of $n = 20$, at $w = 1.5$, it starts at zero since the norm contains Inf and/or NaN, the plots "blow up". This emphasize that the matrix maybe diverging due large matrix size or when the spectral radius $\rho(B_{JOR}) > 1$. This would make sense because the eigenvalues of the iteration matrix for JOR may have grown unstable as shown when $w = 1.5$ when $n = 20$





To test if the method blows up, the spectral radius of the JOR iteration matrix was constructed for $n = 4, 10, 20$ based on $0 < w < 1$. For the different size n on the spectral radius $\rho(B_{JOR})$ blows up after $w = 1$. From 0.1 to 1 it slightly decreases but increases for a high w setting. When n increases it is shown that the method diverges and proves for when n is small, JOR works when $w < 1$. For more on the spectral radius results, Table 6 gives the spectral radii for the different matrix sizes n.

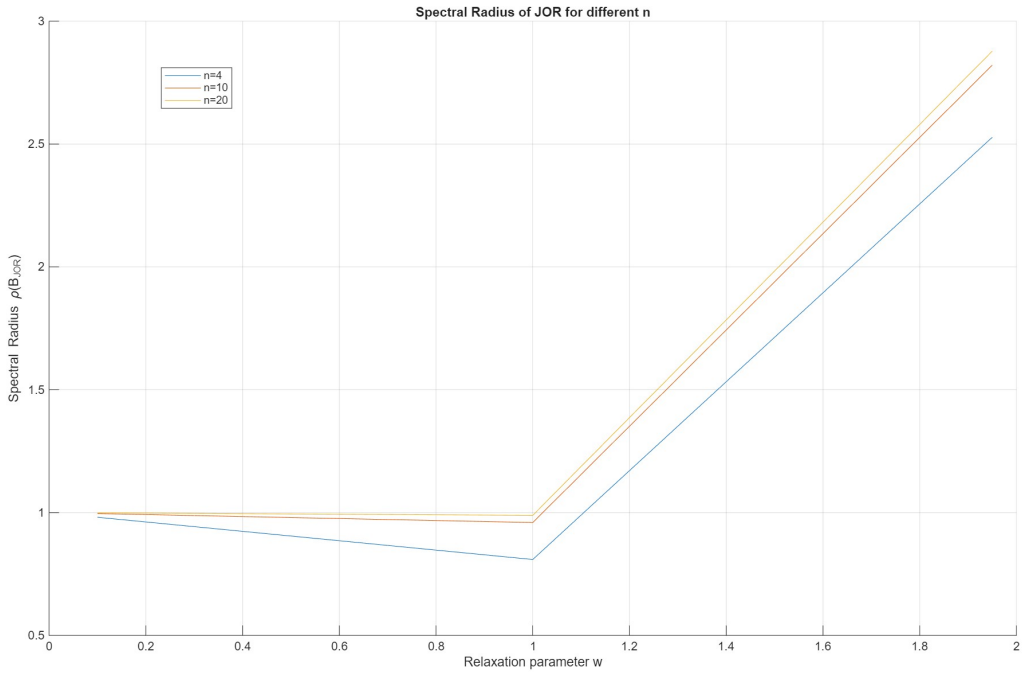


Table 6: Spectral Radius of B_{JOR}

n	w	$\rho(B_{JOR})$
4	1	0.81
10	1	0.96
20	1	0.99

6.3.4 SOR Method

For the SOR method, I will consider the Gauss-Seidel method but with a relaxation parameter w and residual $r_i^{(k)}$ as follows:

$$x_i^{(k+1)} = x_i^{(k)} + \frac{w}{a_{ii}} r_i^{(k)}, \quad r_i^{(k)} = b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^{n} a_{ij} x_j^{(k)}$$

This changes Gauss-Sediel where w acts as a scaler to "help" solve the $Hx = b$ solution.

When w is
- $0 < w < 1$, the method undergoes Under-relaxation leading to slower convergence
- $w = 1$, the method reduces to Gauss-Seidel.
- $1 < w < 21$, the method undergoes Over-relaxation leading to faster convergence

Rewriting it to matrix form,

$$x^{(k+1)} = (D - wL)^{-1}((1-w)D + wU)x^{(k)} + w(D - wL)^{-1}b,$$

so the iteration matrix and constant vector for SOR are:

$$B_{SOR} = (D - wL)^{-1}((1-w)D + wU), \quad c_{SOR} = w(D - wL)^{-1}b.$$

For the solution to the $Hx = b$, by the SOR method, the true solution is a vector of ones based on the matrix H of size n. As for the approximated solution vector $\tilde{x}$ for each of the w ranges in the convergent case has an approximation within the 10^{-10} of the true solution x. For $n = 20$, $\tilde{x}$ is a vector of approximately one. For more convergent behaviors, (Table 7) listed below shows the SOR converges better for when $w = 1.5$ for higher matrices compared to smaller matrices when $w = 0.5$.

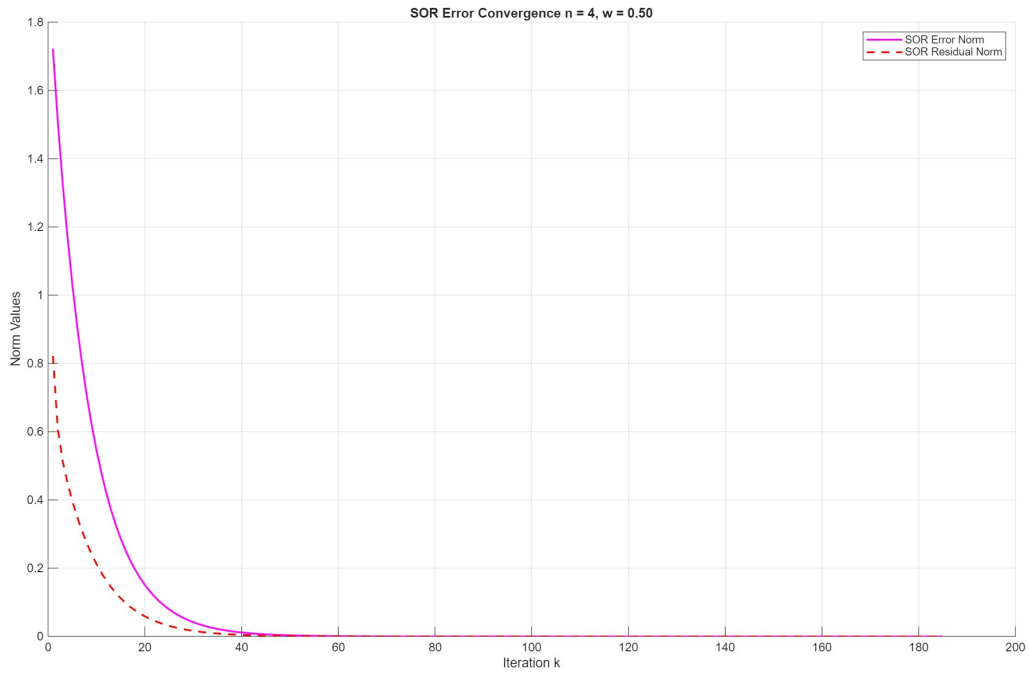
$$x = \left[1, 1, \cdots, 1\right]_{n\times 1}^{T} \quad \text{(SOR true solution } x\text{)}$$

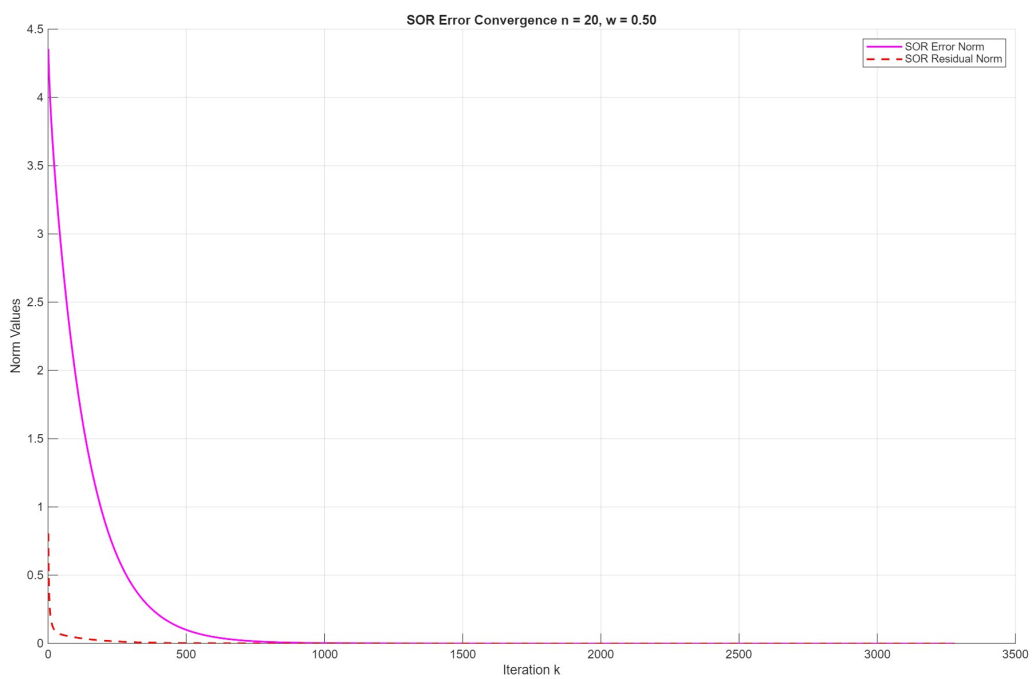
$$\tilde{x}^{(k)} \approx \left[0.999999999999999, 0.999999999999999, \cdots, 0.999999999999999\right]_{20\times 1}^{T}$$

Table 7: SOR Method Performance

n	w	**Iteration Number**	**Final** $\|x^{(k)}\|$	**Final** $\|r^{(k)}\|$
4	0.5	185	$9.19 * 10^{-11}$	$3.58 * 10^{-11}$
20	0.5	3281	$9.87 * 10^{-11}$	$2.23 * 10^{-12}$
22	0.5	3942	$9.98 * 10^{-11}$	$1.86 * 10^{-12}$
4	1.0	57	$7.54 * 10^{-11}$	$3.46 * 10^{-11}$
20	1.0	1089	$9.91 * 10^{-11}$	$2.25 * 10^{-12}$
38	1.0	3811	$1.00 * 10^{-12}$	$6.52 * 10^{-13}$
4	1.5	35	$4.33 * 10^{-11}$	$1.06 * 10^{-10}$
20	1.5	348	$9.50 * 10^{-11}$	$2.44 * 10^{-12}$
67	1.5	3893	$9.78 * 10^{-11}$	$2.16 * 10^{-13}$

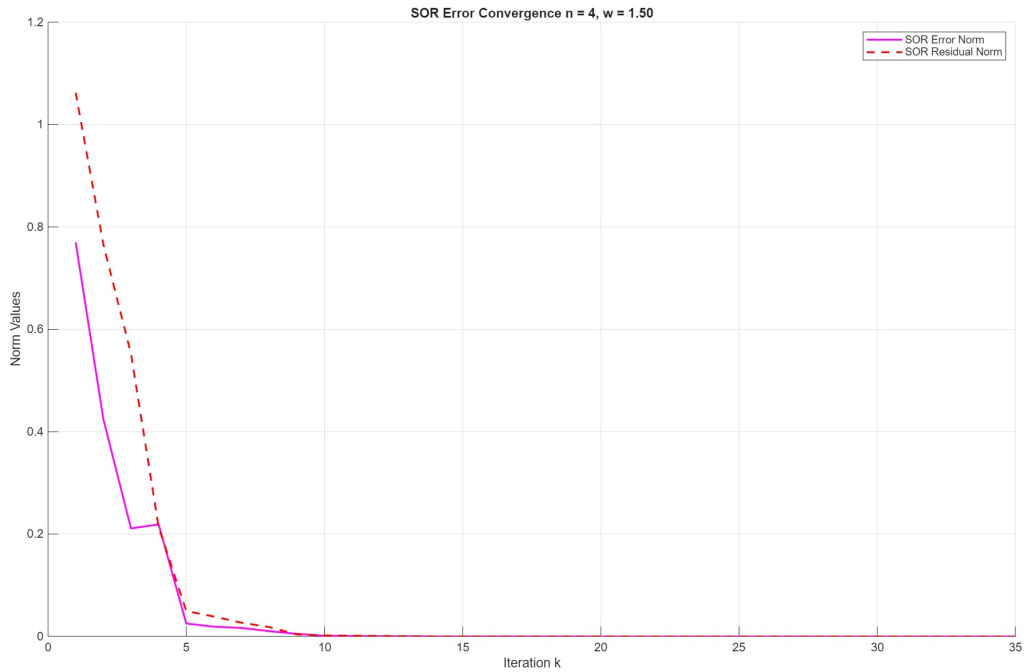
For convergence behaviors with relaxation parameter w. For $0 < w < 1$, it takes longer to converge but as for the matrix size n increase, it breaks at $n = 23$. For $1 < w < 2$, SOR undergoes over-relaxation where w accelerates convergence. There is a huge reduction in the number of iterations it converges and is more efficient up to a matrix size $n = 67$. As for the error and residual norms, its more of the same estimation as with the prior three methods. For $0 < w < 1$, the under-relaxation plots depict a similar plot to Jacobi and Gauss-Sediel. It takes longer to converge but as the matrix size n increase, it breaks at $n = 23$. This would indicate that for a smaller w, the optimal parameter is not the best when $0 < w < 1$.

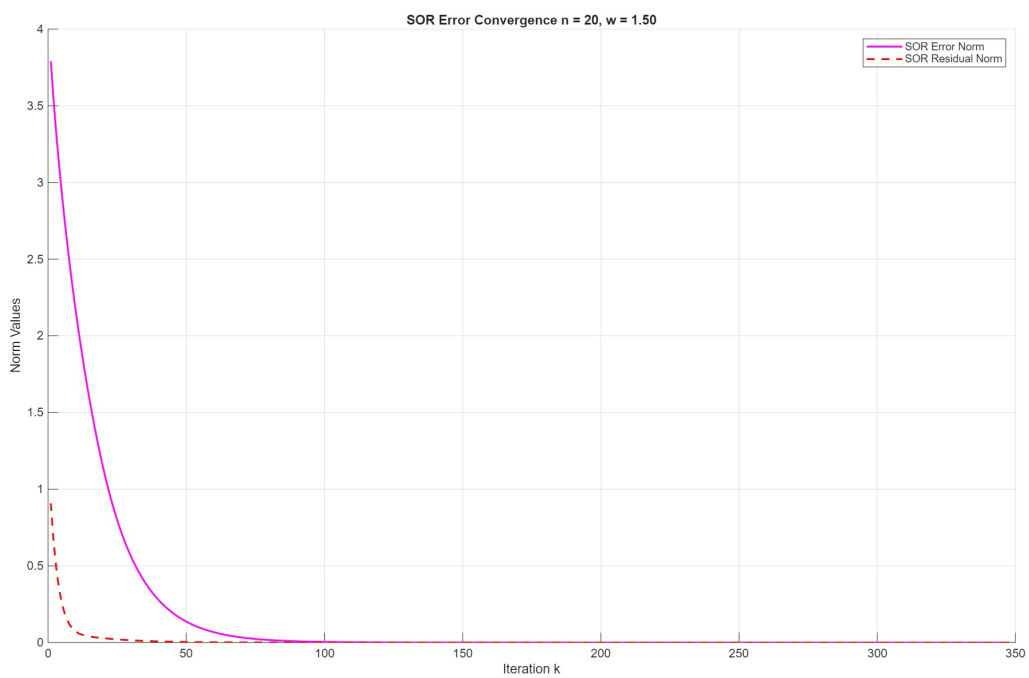




For $1 < w < 2$, SOR undergoes over-relaxation where w accelerates convergence. There is a huge reduction in the number of iterations it converges and is more efficient. When $n = 4$, like the JOR method, the estimation solution error, starts off under the residual norm but then adjusts to the estimation of the residual norm. When adjusting to $n = 20$, there is an improved convergence until it breaks at matrix size $n = 67$. The results demonstrate

that proper selection of w can significantly improve convergence, particularly for moderately sized systems. To further test SOR, I will find the optimal value of relaxation parameter of w when $0 < w < 2$.





Using the spectral radius of the iteration matrix of SOR, B_{SOR} with matrices size $n = 4, 20, 50,$ and 100 the relaxation parameters get close to 2 while the spectral radius

of the iteration matrix, $\rho(B_{SOR}) < 1$ to maintain convergence. A graphical depiction of the parameters shows that the the smallest ρ indicates the optimal w. Based on depiction, the optimal relaxation parameter w slowly spikes and is close to 2. For a guess, I would say $w \approx 1.95$ since for $n = 100$, the lowest $\rho(B_{SOR}) = 0.94$ is at $w = 1.94$. also lists the spectral radii ouptputs on the following page.

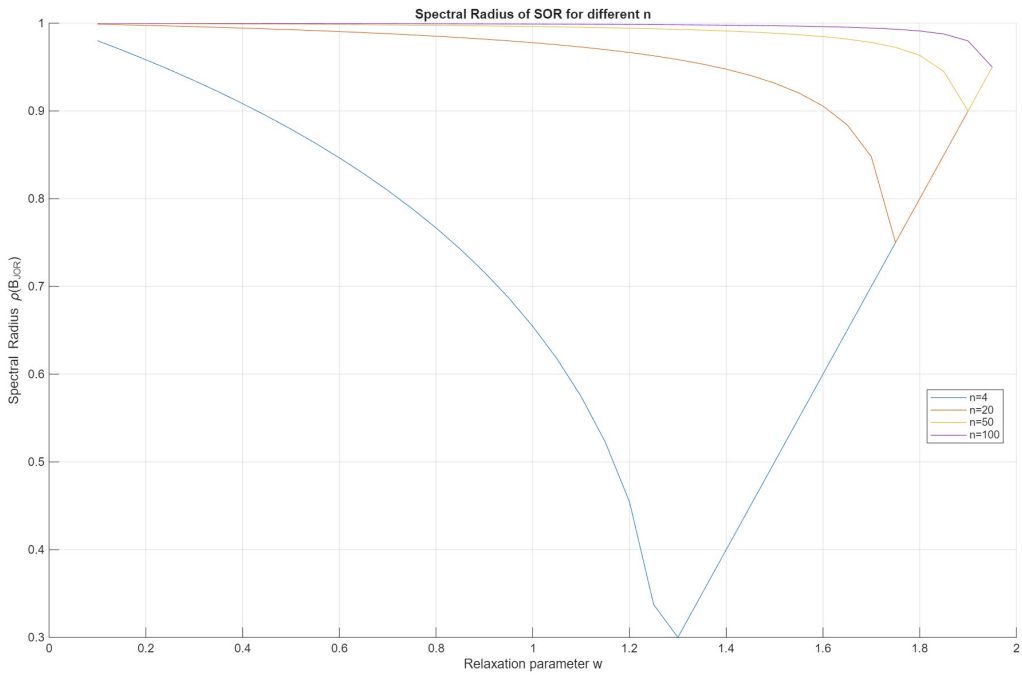


Table 8: Spectral Radius of B_{SOR}

n	w	$\rho(B_{SOR})$
4	1.26	0.30
20	1.75	0.75
50	1.89	0.90
100	1.94	0.95

7 Conclusion

This project examined the numerical behavior of tridiagonal matrices in solving the system $Hx = b$, highlighting both analytical and computational insights. Tridiagonal matrices are proven to b LU factorable and by Gaussian elimination its proven to be highly efficient and consistently accurate across arbitrary matrix size n. Spectral radii analysis further underscored the role of the spectral radius in determining iterative convergence for arbitrary size n. This impacts the approximation to $\tilde{x}^{(k)}$ in matching the true solution $x = \mathbf{1_{n\times 1}}$ to within 10^{-10} and produced residual norms on machine precision.

In terms of convergence, the Jacobi method converges for small matrices under the set conditions ($\epsilon = 10^{-10}$, maximum iterations 4,000), but its performance deteriorates as n increases. Gauss-Seidel converges faster than Jacobi by using updated $x^{(k)}$ values in each iteration which reduces the effective spectral radii and improves error accuracy. As for the JOR it is proven to be unstable, often producing NaN or Inf values for moderate to large matrices. For SOR, under-relaxation ($w < 1$) slows convergence, while over-relaxation ($w > 1$) accelerates it, particularly for larger matrices. When SOR is with $1 < w < 2$, it achieved the fastest performance in contrast to JOR, where it effectively reduces the spectral radius of B_{SOR} to achieve the best approximation to the true solution vector. Based on trends, the optimal w is estimated near $w \approx 1.9$–1.95.

For the error and residual norm polots, each method appear to have similar behaviors. The solution error norm is usually higher than the residual norm and each slowly converges almost always in a logarithm behavior to the set convergence threshold. For JOR method, due to having NaN and Inf entries, it fails to satisfy the convergence threshold. The JOR plots are initially consistent with its approximation analysis but due to the w parameter it hurts or delays the divergence in the approximation. Gauss–Seidel and SOR with $w > 1$ are proven to be the better iterative solvers for a large n. The optimal relaxation parameter for SOR was found numerically to be close to $w = 1.95$, which corresponds to the smallest measured spectral radius. These results illustrate the strong connection between matrix structure, spectral radius, and convergence behavior, and highlight why SOR and Gauss–Seidel remain preferred iterative methods for solving large tridiagonal systems in numerical analysis.

8 Appendices

References

[1] Carl D. Meyer. *Matrix Analysis and Applied Linear Algebra*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, USA, 2000.

8.1 MATLAB Code

```
clc, clearvars

format rat

% create matrix A for n = 4
% A is a tridiagonal matrix where on main diagonal it's all 2's
% and above below the 2's are -1's

n = 4;                          % size of matrix
a = -1 * ones(n-1,1);           % subdiagonal
b =  2 * ones(n,1);             % main diagonal
c = -1 * ones(n-1,1);           % superdiagonal
A = diag(b) + diag(a,-1) + diag(c,1);

% b vector
b = zeros(n,1);
b(1) = 1;
b(end) = 1;

%% eigenvalues
%{
% Different n values to compare
n_values = [4, 10, 20];

figure; hold on;

for i = 1:length(n_values)
    n = n_values(i);
    k = 1:n;
    lambda = 2 - 2*cos(k*pi/(n+1));  % eigenvalues formula
    plot(k, lambda, 'o-', 'DisplayName', ['n=' num2str(n)]);

    fprintf('The spectral radius for n = %d is: %0.2f\n', n, lambda(
        n))
end

xlabel('Eigenvalue index k');
ylabel('\lambda_k');
```

```
title('Eigenvalues of Tridiagonal Matrix');
legend('show');
grid on;
%}

%% check matrix
B =[ [17, 24, 1, 8, 15];
     [23, 5, 7, 14, 16] ;
     [ 4, 6, 13, 20, 22] ;
     [10, 12, 17, 21, 3] ;
     [11, 18, 25, 2, 18] ];



%% Gaussian elimination
function[] = GE(A, b)
% LU factorization w/o pivoting (A = LU)
% A is nxn and has LU factorization

% copy of original matrix
Aog = A;
bog = b;

[n, n] = size(A);
for k = 1:n-1

    % lij; pivot entry
    A(k+1:n, k) = A(k+1:n, k) / A(k,k);
    bm = A(k+1:n, k);

    % make all entries below zero
    A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - A(k+1:n, k) * A(k,k+1:n);
    b(k+1:n) = b(k+1:n) - bm .* b(k);

end

L = eye(n, n) + tril(A, -1);
U = triu(A);

fprintf("With Gaussian Elimination the matrix A is: \n")
disp(U);

%back substitution

% Ax = b; Ux = b -> x = U^-1 *b
x = zeros(n,1);

```

```
% backward substitution
for i = n:-1:1
    x(i) = (b(i) - U(i,i+1:n) * x(i+1:n)) / U(i,i);
end

% output
fprintf("With back substitution with the matrix A and vector where \
   n")
fprintf("A is: \n")
disp(U);

fprintf("and b is: \n")
disp(b);

fprintf("The solution vector to Ax = b, for x is: \n")
disp(x);

format long

%check: Ax = b or b - Ax ~= 0
disp(bog - Aog*x);

end

% GE(A,b)

%% Gaussian elimination with partial pivoting
function[] = PGE(A, b)
% LU factorization with partial pivoting (A = LU)
% A is nxn and has LU factorization

% copy of original matrix
Aog = A;
bog = b;

[n, n] = size(A);
for k = 1:n-1

    % partial pivoting
    [~, pivot] = max(abs(A(k:n,k)));
    pivot = pivot + k - 1;

    if pivot ~= k

        % Swaping Rows
        A([pivot,k], :) = A([k,pivot], :);
        P([pivot,k], :) = P([k,pivot], :);
```

```
        if k > 1
            L([k pivot], 1:k-1) = L([pivot k], 1:k-1);
        end

    end

    % lij; pivot entry
    A(k+1:n, k) = A(k+1:n, k) / A(k,k);
    bm = A(k+1:n, k);

    % make all entries below zero
    A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - A(k+1:n, k) * A(k,k+1:n);
    b(k+1:n) = b(k+1:n) - bm .* b(k);

end

L = eye(n, n) + tril(A, -1);
U = triu(A);

fprintf("With Gaussian Elimination by Partial Pivoting the matrix A is: \n")
disp(U);

%back substitution

% Ax = b; Ux = b -> x = U^-1 *b
x = zeros(n,1);

% backward substitution
for i = n:-1:1
    x(i) = (b(i) - U(i,i+1:n) * x(i+1:n)) / U(i,i);
end

format rat
% output
fprintf("With back substitution with the matrix A and vector where \n")
fprintf("A is: \n")
disp(U);

fprintf("and b is: \n")
disp(b);

fprintf("The solution vector to Ax = b, for x is: \n")
disp(x);
```

```
format long

%check: Ax = b or b - Ax ~= 0
disp(bog - Aog*x);

end

%PGE(A,b)

%% LU factorization without partial pivoting
function[L, U] = LU(A)
% LU factorization w/o pivoting (A = LU)
% A is nxn and has LU factorization

% original A
Aog = A;

[n, n] = size(A);
for k = 1:n-1

    % lij; pivot entry
    A(k+1:n, k) = A(k+1:n, k) / A(k,k);

    % make all entries below zero
    A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - A(k+1:n, k) * A(k,k+1:n);

end

L = eye(n, n) + tril(A, -1);
U = triu(A);

% change format
format rat

fprintf("With LU factorization of A, \n")
fprintf("the Lower triangular matrix, L is: \n");
disp(L);

fprintf("and the Upper triangular matrix, U is: \n");
disp(U);

fprintf("Check: A - LU: \n")
disp(Aog - L*U)

end

% get outputs
```

```
%[L, U] = LU(A);

function[y, x] = sb(L, U , b)
% implement forward and backward subsitituion to solve Ax = b
% sb = substitution (forward then backwards)
% Ax = (LU)x = Ly = b where y = Ux;
% solve Ly = b for y; then Ux = y for x
% do with code from P8

% get origional b vector
bog = b;

% size of b vector
n = size(bog, 1);

% Ly = b; y = L^-1 * b
y = zeros(n,1);

% forward subsitution
for i = 1:n
    y(i) = (b(i) - L(i,1:i-1) * y(1:i-1)) / L(i,i);
end

% Ux = y; x = U^-1 * y
x = zeros(n,1);

% backward substitution
for i = n:-1:1
    x(i) = (y(i) - U(i,i+1:n) * x(i+1:n)) / U(i,i);
end

fprintf("For forward substitution, y is:\n");
disp(y);

fprintf("For back subsutition,\n");
fprintf("The solution vector x: \n")
disp(x);

% check
format long

fprintf("Check matrix Ax = b; b - Ax ~= 0 \n")
A = L*U;
disp(bog - A*x)

end

```

```
% output
%[y, x] = sb(L, U , b)

%% Iterative Methods

eigenvalues = eigs(A);
eigenvalues;
format short


function JM(A, b)
% Jacobi method to approximate a solution to Ax = b
% A = matrix (n, n)
% b = solution vector (n x 1)
% w = relaxation parameter

% initialize
n = size(A,1);      % size of the matrix

% initialize D, L and U. A = D - L - U
D = diag(diag(A));
L = -tril(A,-1);
U = -triu(A,1);

% Jacobi
% B = Bjac and c = cjac
Bjac = D^-1 * (L + U);
cjac = D^-1 * b;

% initialize x, convergence and norms
xJ = zeros(n,1);
epsilon = 1e-10;
maxIter = 4000;
EnormJ = zeros(n,1);
RnormJ = zeros(n,1);

% exact matrix and residual
x_exact = A \ b;

% Track convergence iterations
convJ = maxIter;

% method iteration
for k = 1:maxIter  % set iteration to something (you do it)

   % Jacobi
   % x(k+1) = Bjac*x(k) + cjac
```

```
    xNewJ = Bjac * xJ + cjac;

     % calculate Error norms
    EnormJ(k) = norm(x_exact - xNewJ);

    % Residual norm
    RnormJ(k) = norm(b - A*xNewJ);

    % convergence
     if EnormJ(k) < epsilon && convJ == maxIter
         convJ = k;
         fprintf('Jacobi converged at iteration %d\n', k);
     end

    % change after each loop
    xJ = xNewJ;

end

% get norm and k values for plot
lastIter = max([convJ]);
kVals = (1:lastIter)';

EnormJ  = EnormJ(1:lastIter);
RnormJ  = RnormJ(1:lastIter);

% print table:
T = table(kVals, EnormJ, RnormJ, ...
          'VariableNames', {'Iteration', 'JErrorNorm', 'JResidualNorm'});

disp(T)

% plot k vs error norm
figure;
hold on;
semilogy(kVals, EnormJ, '-b','LineWidth',1.5);
semilogy(kVals, RnormJ, '--r','LineWidth',1.5);
xlabel('Iteration k');
ylabel('Norm Values');
title(sprintf('Jacobi Error Convergence, n = %d', n));
grid on;
legend('Jacobi Error Norm','Jacobi Residual Norm', 'Location', 'northeast');

fprintf('The approximated solutions for Jacobi is: \n')
format long
```

```
disp(xJ)

end
%JM(A, b)

%[kValsGS, EnormGS, RnormGS] =
function GS(A, b)
% Gauss Seidal method to approximate a solution to Ax = b
% A = matrix (n, n)
% b = solution vector (n x 1)

% initialize
n = size(A,1);      % size of the matrix

% initalize D, L and U. A = D - L - U
D = diag(diag(A));
L = -tril(A,-1);
U = -triu(A,1);

% B = Bgs and c = cgs
Bgs = (D - L)^-1 * U;
cgs = (D - L)^-1 * b;

% initialize x, convergence and norms
xGS = zeros(n,1);
epsilon = 1e-10;
maxIter = 4000;
Enorm = zeros(n,1);
Rnorm = zeros(n,1);

% exact matrix
x_exact = A \ b;

% Track convergence iterations
convGS = maxIter;

% Gauss Seidal method
for k = 1:maxIter  % set iteration to something (you do it)

    % x(k+1) = Bgs*x(k) + cgs
    xNew = Bgs * xGS + cgs;

    % residual
    rGS = b - A*xNew;

    % calculate norms
    Enorm(k) = norm(x_exact - xNew);
```

```
    Rnorm(k) = norm(rGS);

    % convergence
    if Enorm(k) < epsilon && convGS == maxIter
        convGS = k;
        fprintf('Gauss-Seidel converged at iteration %d\n', k);
    end

    % change after each loop
    xGS = xNew;

end

% get norm and k values for plot
lastIter = max([convGS]);
kValsGS = (1:lastIter)';

EnormGS  = Enorm(1:lastIter);
RnormGS  = Rnorm(1:lastIter);

% print table:
T = table(kValsGS, EnormGS, RnormGS, ...
          'VariableNames', {'Iteration', 'ErrorNorm', 'ResidualNorm'
             });

disp(T)

% Plot k vs residual norm
figure;
hold on;
semilogy(kValsGS, EnormGS, '-g','LineWidth',1.5);
semilogy(kValsGS, RnormGS, '--r','LineWidth',1.5);
xlabel('Iteration k');
ylabel('Norm Values');
title(sprintf('Gauss Seidal Error Convergence, n = %d', n));
legend('Error Norm', 'Residual Norm', ...
   'Location', 'northeast');
grid on;

fprintf('The approximated solutions for Gauss-Sediel is: \n')
format long
disp(xGS)

end
%GS(A, b)

function JOR(A, b, w)
```

```
% JOR method to approximate a solution to Ax = b
% A = matrix (n, n)
% b = solution vector (n x 1)
% w = relaxation parameter

% initialize
n = size(A,1);      % size of the matrix

% initialize D, L and U. A = D - L - U
D = diag(diag(A));
L = -tril(A,-1);
U = -triu(A,1);

% JOR
% B = Bjac and c = cjac
Bjor = D^-1 * (w*(L + U) + (1-w)*D );
cjor = w * D^-1 * b;

% initialize x, convergence and norms
xJOR = zeros(n,1);
epsilon = 1e-10;
maxIter = 4000;
EnormJOR = zeros(n,1);
RnormJOR = zeros(n,1);

% exact matrix and residual
x_exact = A \ b;

% Track convergence iterations
convJOR = maxIter;

% method iteration
for k = 1:maxIter  % set iteration to something (you do it)

   % Jacobi
   % x(k+1) = Bjac*x(k) + cjac
   xNewJOR = Bjor * xJOR + cjor;

    % calculate Error norms
   EnormJOR(k) = norm(x_exact - xNewJOR);

   % Residual norm
   RnormJOR(k) = norm(b - A*xNewJOR);

   % convergence
    if EnormJOR(k) < epsilon && convJOR == maxIter
        convJOR = k;
```

```
        fprintf('JOR converged at iteration %d\n', k);
    end

   % change after each loop
   xJOR = xNewJOR;

end

% get norm and k values for plot
lastIter = max([convJOR]);
kVals = (1:lastIter)';

EnormJOR  = EnormJOR(1:lastIter);
RnormJOR  = RnormJOR(1:lastIter);

% print table:
T = table(kVals, EnormJOR, RnormJOR, ...
          'VariableNames', {'Iteration', 'JORErrorNorm', 'JORResidualNorm'});

disp(T)

% plot k vs error norm
figure;
hold on;
semilogy(kVals, EnormJOR, '-b','LineWidth',1.5);
semilogy(kVals, RnormJOR, '--r','LineWidth',1.5);
xlabel('Iteration k');
ylabel('Norm Values');
title(sprintf('JOR Error Convergence n = %d, w = %0.02f', n, w));
grid on;
legend('JOR Error Norm','JOR Residual Norm', 'Location', 'northeast');

fprintf('The approximated solutions for JOR is: \n')
format long
disp(xJOR)

end
%JOR(A,b,0.5)
%JOR(A,b,1.0)
%JOR(A,b,1.5)

%% Bjor estimate for w
n_values = [4, 10, 20];
w_values = 0.1:0.05:1.95;

```

```
figure; hold on;
colors = lines(length(n_values));

for j = 1:length(n_values)
    n = n_values(j);
    A = 2*eye(n) - diag(ones(n-1,1),1) - diag(ones(n-1,1),-1);
    D = diag(diag(A));
    L = -tril(A,-1);
    U = -triu(A,1);

    rho_vals = zeros(size(w_values));
    for i = 1:length(w_values)
        w = w_values(i);
        Bjor = D^-1 * ((1-w)*D + w*(L + U));
        rho_vals(i) = max(abs(eig(Bjor)));
    end
    plot(w_values, rho_vals, '-','Color',colors(j,:),'DisplayName',
       sprintf('n=%d', n));

    % Find w that minimizes spectral radius
    [~, k] = min(rho_vals);
    w_opt = w_values(k);
    fprintf('n = %d: Optimal w ~ %.3f with spectral radius %.2f\n',
       n, w_opt, rho_vals(k));

end

xlabel('Relaxation parameter w');
ylabel('Spectral Radius \rho(B_{JOR})');
title('Spectral Radius of JOR for different n');
grid on;
legend show;

function SOR(A, b, w)
% SOR method to approximate a solution to Ax = b
% A = matrix (n, n)
% b = solution vector (n x 1)
% w = relaxation parameter

% initialize
n = size(A,1);      % size of the matrix

% initialize D, L and U. A = D - L - U
D = diag(diag(A));
L = -tril(A,-1);
U = -triu(A,1);

```

```
% SOR
% B = Bsor and c = csor
Bsor = (D - w*L)^-1 * ( (1-w)*D + w*U );
csor = w*(D - w*L)^-1 * b;

% initialize x, convergence and norms
xSOR = zeros(n,1);
epsilon = 1e-10;
maxIter = 4000;
EnormSOR = zeros(n,1);
RnormSOR = zeros(n,1);

% exact matrix and residual
x_exact = A \ b;

% Track convergence iterations
convSOR = maxIter;

% method iteration
for k = 1:maxIter  % set iteration to something (you do it)

   % SOR
   % x(k+1) = Bsor*x(k) + csor
   xNewSOR = Bsor * xSOR + csor;

    % calculate Error norms
   EnormSOR(k) = norm(x_exact - xNewSOR);

   % Residual norm
   RnormSOR(k) = norm(b - A*xNewSOR);

   % convergence
    if EnormSOR(k) < epsilon && convSOR == maxIter
        convSOR = k;
        fprintf('SOR converged at iteration %d\n', k);
    end

   % change after each loop
   xSOR = xNewSOR;

end

% get norm and k values for plot
% Slice vectors to actual iterations
lastIter = max([convSOR]);
kValsSOR = (1:lastIter)';

```

```
EnormSOR = EnormSOR(1:lastIter);
RnormSOR = RnormSOR(1:lastIter);

% print table:
T = table(kValsSOR, EnormSOR, RnormSOR, ...
          'VariableNames', {'Iteration', 'SORErrorNorm', 'SORResidualNorm'});

disp(T)

% plot k vs error norm
figure;
hold on;
semilogy(kValsSOR, EnormSOR, '-m','LineWidth',1.5);
semilogy(kValsSOR, RnormSOR, '--r','LineWidth',1.5);
xlabel('Iteration k');
ylabel('Norm Values');
title(sprintf('SOR Error Convergence n = %d, w = %0.02f', n, w));
grid on;
legend('SOR Error Norm', 'SOR Residual Norm', 'Location', 'northeast');

fprintf('The approximated solutions for SOR is: \n')
format long
disp(xSOR)

end

% for SOR Method; 0 <= w < 1; w = 0.5, 1.5
%SOR(A, b, 0.5)
%SOR(A, b, 1.0)
%SOR(A, b, 1.5)

%% Bsor estimate for w

n_values = [4, 20, 50, 100];
w_values = 0.1:0.05:1.99;

figure; hold on;
colors = lines(length(n_values));

for j = 1:length(n_values)
    n = n_values(j);
    A = 2*eye(n) - diag(ones(n-1,1),1) - diag(ones(n-1,1),-1);
    D = diag(diag(A));
    L = -tril(A,-1);
    U = -triu(A,1);
```

```
    rho_vals = zeros(size(w_values));
    for i = 1:length(w_values)
        w = w_values(i);
        Bsor = (D - w*L) \ ((1-w)*D + w*U);
        rho_vals(i) = max(abs(eig(Bsor)));
    end
    plot(w_values, rho_vals, '-','Color',colors(j,:),'DisplayName',
       sprintf('n=%d', n));

    % Find w that minimizes spectral radius
    [~, k] = min(rho_vals);
    w_opt = w_values(k);
    fprintf('n = %d: Optimal w ~ %.3f with spectral radius %.2f\n',
       n, w_opt, rho_vals(k));
end

xlabel('Relaxation parameter w');
ylabel('Spectral Radius \rho(B_{JOR})');
title('Spectral Radius of SOR for different n');
grid on;
legend show;
```